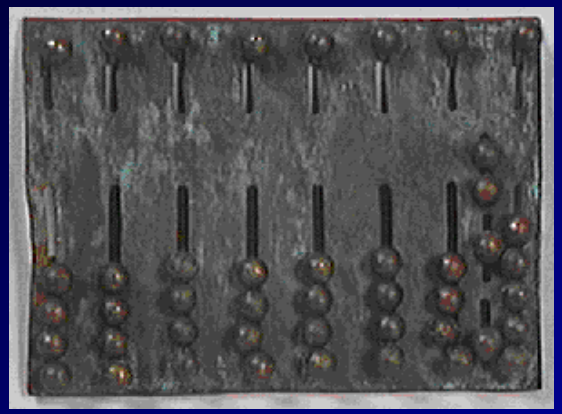
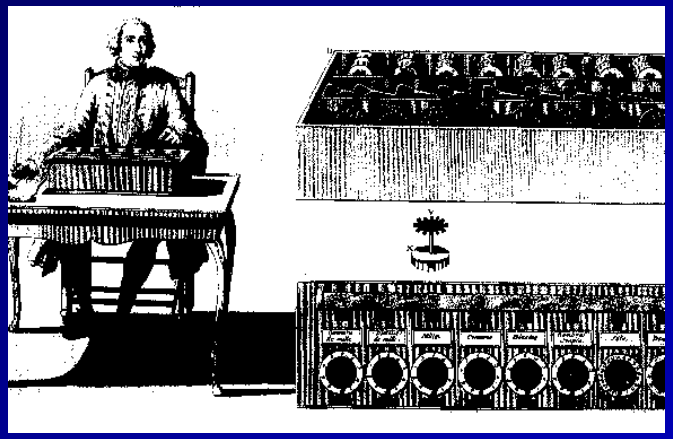
Architecture des Ordinateurs







Ecole Supérieure de Gestion d'Informatique et des Sciences

Objectifs du Cours :

- Comprendre la structure interne d'un ordinateur ;
- Comprendre les principes de fonctionnement des ordinateurs ;
- Décrire le rôle des composants de l'ordinateur ;
- Analyser le fonctionnement d'un processeur et les interactions entre le processeur et son environnement.

PROGRAMME :

I - REPRESENTATION DES DONNEES

- Codage / binaire
- Changements de base
- Représentation des caractères

II - CIRCUITS LOGIQUES : ALGEBRE DE BOOLE

III - ARCHITECTURE DE BASE

- Principe de fonctionnement
- Processeur/Mémoire principale /Entrées-Sorties

INTRODUCTION :

L'informatique, est la science du traitement de l'information. Apparue au milieu du 20ème siècle, elle a connu une évolution extrêmement rapide. A la motivation initiale qui était de faciliter et d'accélérer le calcul, se sont ajoutées de nombreuses fonctionnalités, comme l'automatisation, le contrôle et la commande de processus, la communication ou le partage de l'information.

L'architecture d'un système représente l’organisation de ses différentes unités et de leurs interconnexions.
L’architecture d’un ordinateur signifie comment est fait un ordinateur, quels sont les éléments qui le constituent, quel est son mode de fonctionnement et comment l’information circule et est traitée par les différents constituants de l’ordinateur.
Cette étude abordera le processeur et son langage machine, les fonctions de base de son système (BIOS), et ses mécanismes de communication avec l’extérieur (entrées/sorties).

I - REPRÉSENTATION DES DONNÉES :

Pour traiter l'information, il faut la représenter sous une forme manipulable par le système de traitement. Le codage d'une information consiste à établir une correspondance entre la représentation externe de l'information (exemple le caractère A ou le nombre 36), et sa représentation interne dans la machine.

1 – Codage et Opérations Binaires

Les informations traitées par un microprocesseur sont de différents types (nombres, instructions, images, vidéo, etc.…) mais elles sont toujours représentées sous un format binaire.

On utilise la représentation binaire car elle est simple, facile à réaliser techniquement à l'aide de bistables (système à deux états réalisés à l'aide de transistors).

En binaire, une information élémentaire est appelé bit et ne peut prendre que deux valeurs différentes : 0 ou 1.
Une information plus complexe sera codée sur plusieurs bits. On appelle cet ensemble un mot.
Un mot de 8 bits est appelé un octet.

a) Représentation d'un nombre entier en binaire :

Les nombres sont exprimés par des chiffres pouvant prendre deux valeurs 0 ou 1. A chaque chiffre est affecté un poids exprimé en puissance de 2.
Ex : $(101)_2 = 1x\ 2^2 + 0x2^1 + 1x2^0 = (5)_{10}$

b) Représentation d'un nombre entier en hexadécimal

Lorsqu'une donnée est représentée sur plus de 4 bits, on préfère souvent l'exprimer en hexadécimal.
Les nombres sont exprimés par des chiffres et des lettres pouvant prendre 16 valeurs : 0 1 2 3 4 5 6 7 8 9 A B C D E F
A chaque chiffre est affecté un poids exprimé en puissance de 16.

Ex : $(9A)_{16} = 9x16^1 + Ax16^0 = 9x16 + 10x1 = (154)_{10}$

c) Utilisation d'une base quelconque - changement de base

Un nombre N en base B s'écrit :

$N_B = a_n a_{n-1} a_{n-2} \ldots a_1 a_0$ avec $0 \leq a_i \leq B-1$

Exemples :

base 2 (binaire) : 01101110010_2 $0 \leq a_i \leq 1$

base 8 (octal) : 1245676470620_8 $0 \leq a_i \leq 7$

base 10 (décimal) : 974682056_{10} $0 \leq a_i \leq 9$

base 16 (hexadécimal) : $3A5E8F_{16}$ $0 \leq a_i \leq F$

chiffres hexadécimaux : 0 1 2 3 4 5 6 7 8 9 A B C D E F

Pour passer d'une base à une autre il est toujours possible de passer par la base 10
Pour passer du binaire à une base de type 2^n, il suffit de regrouper les bits par n de la droite vers la gauche afin de retrouver chaque chiffre en base 2^n.

Exemples : 10110111_2 à représenter

en octal ($8 = 2^3$) : $10\ \ 110\ \ 111_{\ 2} = 267_8$
en hexadécimal ($16 = 2^4$) : $1011\ \ 0111_{\ 2} = B7_{16}$

Exercices :

1) On donne le nombre 32745 en base 8. Comment s'écrit-il en base 16 ?

   a) passer par l'intermédiaire de la base 2

   b) passer par l'intermédiaire de la base 10

2) Un nombre s'écrit 753 en base 8. Comment s'écrit-il en base 16 ?

3) Effectuer : $1110101_2 + 11000011_2$

n bits permettent de représenter 2^n valeurs différentes, donc les nombres entiers positifs de 0 à 2^n-1.
Exemple : sur 8 bits on peut représenter $2^8 = 256$ valeurs différentes donc les entiers de 0 à 255 (0000 0000 à 1111 1111).
EXEMPLES :
4 bits
6 bits
8 bits
Généralement :
Avec n bits on représente les nombres entiers de -2^{n-1} à $+2^{n-1} - 1$.

d) Représentation des nombres binaires signés

La solution la plus simple consiste à réserver un bit pour ce signe. Les autres bits représentant une valeur absolue du nombre.

La convention retenue impose de mettre le premier bit (qu'on appelle aussi bit de poids fort) à 0 pour repérer un nombre positif et à 1 pour un nombre négatif.

On parle alors de donnée signée quand on utilise cette convention.

1 1100 = -12

0 1100 = 12

Toutefois, une telle représentation des nombres signés entraînerait un traitement spécial du signe et des circuits électroniques.
Cet inconvénient est résolu par une autre forme de représentation des nombres négatifs dite représentation en complément à 2 (C2) ou encore représentation sous forme complémentée.

- ☞ Pour un nombre entier positif, sa représentation en complément à 2, c'est sa représentation en base 2.

☞ Pour un nombre entier négatif, on obtient sa représentation en complément à 2 en 3 étapes :

1- calculer la représentation binaire de sa valeur absolue

2- complémenter tous les bits (0→1, 1→0)

3- ajouter 1 (additionner 1 au nombre obtenu)

Exercices : -23 sur 8 bits

-100 sur 8 bits

NB :

- La représentation C2 n ’a de sens que si on précise qu’on l ’utilise pour ce nombre. [-23 en C2 vaut 233 en binaire].

- Lorsque des opérations sont effectuées sur des nombres codés en complément à 2, ceux-ci doivent impérativement être codés sur le même nombre de bits.

- Le bit de poids fort d'un nombre négatif est à 1, le bit de poids fort d'un nombre positif est à 0.

Exercices : sur 4 bits

4	0100	4	0100
6	0110	-2	1110

Comment détecter un débordement ?

si les nombres sont de signes différents, pas de Problème ;

S'ils sont de même signe, le signe du résultat doit être le même, sinon il y a débordement.

Exercice :

On considère ces opérations écrites en base 10 :

a) – 61 – 44

b) – 61 – 72

c) 99 – 35

d) 99 + 35

On dispose d'une machine travaillant sur des nombres binaires de longueur 8 (8 bits).

Faire manuellement ce que l'additionneur de la machine ferait automatiquement, et donner les résultats obtenus en binaire. En cas d'erreur, indiquer pourquoi

NB :

1 - Complément à deux : addition, signes opposés

Le résultat est toujours représentable

S'il n' y a pas de retenue, on lit le résultat directement. S'il y a une retenue, on la néglige

2 - Addition de deux nombres de même signe

L'addition de deux nombres de même signe peut donner lieu à un dépassement de capacité !

a) Cas de deux entiers de signe positif :

On a un dépassement de capacité quand la retenue est distincte du dernier bit de report (i.e. celui sur le bit de signe).

b) Cas de deux entiers de signe négatif :

On a toujours une retenue, que l'on oublie.

On a un dépassement de capacité quand la retenue est distincte du dernier bit de report (i.e. celui sur le bit de signe).

e) Représentation des nombres réels

Soit par exemple le nombre fractionnaire 12,346

En décimal il s'écrit :

$12,346 = 1x10^{1} + 2x10^{0} + 3\ x10^{-1} + 4\ x10^{-2} + 6\ x10^{-3}$

Pour la partie entière, on procède par divisions comme pour un entier.

On multiplie la partie fractionnaire par la base en conservant la partie entière chaque fois et on répète l'opération sur la partie fractionnaire du produit jusqu'a ce qu'elle soit nulle (ou que la précision voulue soit atteinte).

Exemple : conversion de $(54,25)_{10}$ en base 2

- Partie entière : $(54)_{10} = (110110)_2$ par divisions.
- Partie fractionnaire :

  $0,25 \times 2 = 0,50 \Rightarrow a^{-1} = 0$

  $0,50 \times 2 = 1,00 \Rightarrow a^{-2} = 1$

  $0,00 \times 2 = 0,00 \Rightarrow a^{-3} = 0$

  $(54,25)_{10} = (110110,010)_2$

Exercices d'application :

$(73,55)_{10} = (\quad)_2$

$(312,3)_4 = (\quad)_8$

Additionner 7,25 et 2,5 en binaire

Convertir les nombres suivants de leur base vers les 3 autres :

10	2	8	16
362,3125			
	10111101,101		
		326,5	
			F3C7,A

CODAGE EN VIRGULE FIXE ET VIRGULE FLOTTANTE :

Soit le nombre décimal 3,25

CODAGE EN VIRGULE FIXE :

Etant donné une base b, un nombre X est représenté en format virgule fixe, par :

$X = a_{n-1}a_{n-2}\ldots a_1a_0,a_{-1}a_{-2}\ldots a_{-p}$ (b)

a_{n-1} est le chiffre de poids fort (MSB)

a_{-p} est le chiffre de poids faible (LSB)

n est le nombre de chiffre avant la virgule

p est le nombre de chiffre après la virgule

CODAGE EN VIRGULE FLOTTANTE :

On a adopté une notation scientifique appelée format à virgule flottante : $x = \pm \mathbf{M} \cdot \mathbf{2}^{\mathbf{E}}$

La notation flottante comprend trois (3) composantes :

- le signe
- le significande (appelé aussi mantisse)
- l'exposant de la puissance

Exemple :

$(3,25)_{10} = 11,01_{(2)}$ (en virgule fixe)

$= 1,101 . 2^{1}$

$= 110,1 . 2^{-1}$

La notation est dite flottante car elle consiste à faire glisser les nombres significatifs à droite ou à gauche

☞Pb : différentes manières de représenter E et M

d'où la nécessité de Normalisation

Standard IEEE 754 (1985 / 2008)

NOTATION :

Les nombres réels sont normalisés sous la forme

- 1,… x 2^n puis codés sur 32bits.
- le 1er bit code le signe (0 pour + et 1 pour -)
- les 8 bits suivants codent l'exposant **auquel on ajoute 127** (permet de coder des exposants négatifs sans C2)
- les 23 derniers bits codent la partie fractionnaire du nombre (après la virgule) mais si le 24ième bit est 1 alors on arrondit à la valeur supérieure (mantisse + 1)

- Simple précision sur 32 bits :

  1 bit de signe de la mantisse

  8 bits pour l'exposant

  23 bits pour la mantisse



1 bit 8 bits 23 bits

- Double précision sur 64 bits :

  1 bit de signe de la mantisse

  11 bits pour l'exposant

  52 bits pour la mantisse



1 bit 11 bits 52 bits

Exemple :

5,3→ 101,0100110011001100110011001100110011001…

Normalisation :

1,0101001100110011001100110011001100110011001.. * 2^2

signe : 0

Exposant : 2 + 127 = 129 soit sur 8 bits 1000 0001

mantisse sur 23 bits : 01010011001100110011001

mais le 24ième bit est 1 donc la mantisse est

01010011001100110011010

codage final :

0100 0000 1010 1001 1001 1001 1001 1010

de manière plus lisible en hexadécimal :

40 A9 99 9A

Remarques :

l'exposant 0000 0000 signifie que le nombre est dénormalisé

l'exposant 1111 1111 signifie que ce n'est pas un nombre (NaN)

En simple précision, les exposants sont donc compris entre -126 et 127

Exercice :

Convertir le nombre décimal 8,625 en virgule flottante suivant la norme IEEE 754 :

8 = 1000

0,625 x2 =1,25 $a^{-1} = 1$

0,25 x2 = 0,50 $a^{-2} = 0$

0,50 x2 = 1,00 $a^{-3} = 1$

8,625 = 1000,101 = 1000,101 x 2^0

= 0,1000101x2^4 (Normalisation)

= 1,000101x2^3 (Pseudo-normalisation IEEE 754)

Signe	Exposant biaisé	Pseudo-mantisse

e) Décimal Codé Binaire et Code de Gray

Le Décimal Codé Binaire ou Binary Coded Decimal (BCD), permet de coder des nombres entiers positifs. En BCD, on code chaque chiffre du nombre décimal sur 4 bits et on concatène ces demi-octets.
Exemple : 4236 devient 0100 0010 0011 0110 en BCD

Le code de Gray, ou code Gray ou code binaire réfléchi est un codage binaire permettant de ne modifier qu'un seul bit à la fois quand un nombre est augmenté d'une unité. Cette propriété est importante pour plusieurs applications.

Codage décimal	Codage binaire	Codage Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

Une méthode consiste à calculer le OU EXCLUSIF entre le binaire de départ et ce même binaire décalé d'un rang à droite

Exercice :

$(5\ 7\ 6)_{10} = (\qquad)_{BCD}$

$(9\ 9)_{10} = (\qquad)_{BCD}$

$(1000\ 0011\ 0110)_{BCD} = (\qquad)_{10}$

Exercice :

Représenter 10 ; 15 et 34 en code Gray

2 - Représentation des caractères

Les caractères sont des données non numériques : il n'y a pas de sens à additionner ou multiplier deux caractères. Par contre, il est souvent utile de comparer deux caractères, par exemple pour les trier dans l'ordre alphabétique.

Le codage des caractères est fait par une table de correspondance indiquant la configuration binaire représentant chaque caractère (ASCII).

DEUX STANDARDS :

- Table ASCII Etendu

  8 bits pour représenter 256 caractères (0 à 255)

  Code les caractères accentues : à, é,…etc.

  Compatible avec ASCII

- Code Unicode (mis au point en 1991)

  16 bits pour représenter 65 536 caractères

  Compatible avec ASCII

  Code la plupart des alphabets : Arabe, Chinois, …

  On a défini environ 50 000 caractères à ce jour

Table ASCII standard (codes de caractères de 0 à 127)

000		(nul)	016	▶	(dle)	032	sp	048	0	064	@	080	P	096	`	112	p
001	☺	(soh)	017	◀	(dc1)	033	!	049	1	065	A	081	Q	097	a	113	q
002	☻	(stx)	018	↕	(dc2)	034	"	050	2	066	B	082	R	098	b	114	r
003	♥	(etx)	019	‼	(dc3)	035	#	051	3	067	C	083	S	099	c	115	s
004	♦	(eot)	020	¶	(dc4)	036	$	052	4	068	D	084	T	100	d	116	t
005	♣	(enq)	021	§	(nak)	037	%	053	5	069	E	085	U	101	e	117	u
006	♠	(ack)	022	▬	(syn)	038	&	054	6	070	F	086	V	102	f	118	v
007	•	(bel)	023	↨	(etb)	039	'	055	7	071	G	087	W	103	g	119	w
008	◘	(bs)	024	↑	(can)	040	(	056	8	072	H	088	X	104	h	120	x
009		(tab)	025	↓	(em)	041	)	057	9	073	I	089	Y	105	i	121	y
010		(lf)	026		(eof)	042	*	058	:	074	J	090	Z	106	j	122	z
011	♂	(vt)	027	←	(esc)	043	+	059	;	075	K	091	[	107	k	123	{
012	♀	(np)	028	∟	(fs)	044	,	060	<	076	L	092	\	108	l	124	|
013		(cr)	029	↔	(gs)	045	-	061	=	077	M	093	]	109	m	125	}
014	♫	(so)	030	▲	(rs)	046	.	062	>	078	N	094	^	110	n	126	~
015	☼	(si)	031	▼	(us)	047	/	063	?	079	O	095	_	111	o	127	⌂

CODE ISO-8859 -1 (Iso-latin-1)

↗	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STH	ETH	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	CD2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL
8	€	□	‚	ƒ	„	…	†	‡	ˆ	‰	Š	‹	Œ	□	Ž	□
9	□	'	'	"	"	•	–	—	˜	™	š	›	œ	□	ž	Ÿ
A		¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
B	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

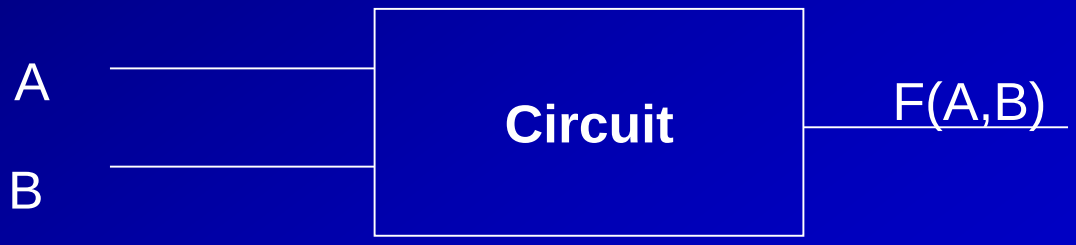
Exercices :

1. En utilisant le code ASCII, écrire votre nom et votre prénom sans oublier de mettre les initiales en majuscule.

2. En utilisant le code ASCII, décoder la phrase suivante : 76 101 32 98 105 110 97 105 114 101 32 99 96 101 115 116 32 102 97 99 105 108 101 46

II – CIRCUITS LOGIQUES – ALGEBRE DE BOOLE

Tous les systèmes numériques (ordinateurs, tablettes, portables, lecteurs CD / DVD, électroménagers programmables, etc.…) sont constitués d'un ensemble de circuits électroniques (portes logiques). Chaque circuit fournit une fonction logique bien déterminée (addition, comparaison,….).



Une seule porte logique (répliquée des milliers ou millions de fois) suffit pour réaliser n'importe quelle machine numérique.

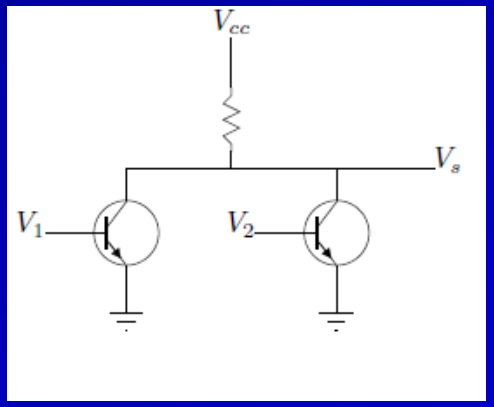
En mathématique, une fonction logique ou opérateur logique est une application de B^n dans B.

En électronique, une fonction logique est une sorte de *boîte noire* qui reçoit en entrée un certain nombre de variables logiques et qui rend en sortie une variable logique dépendant des variables d'entrée.

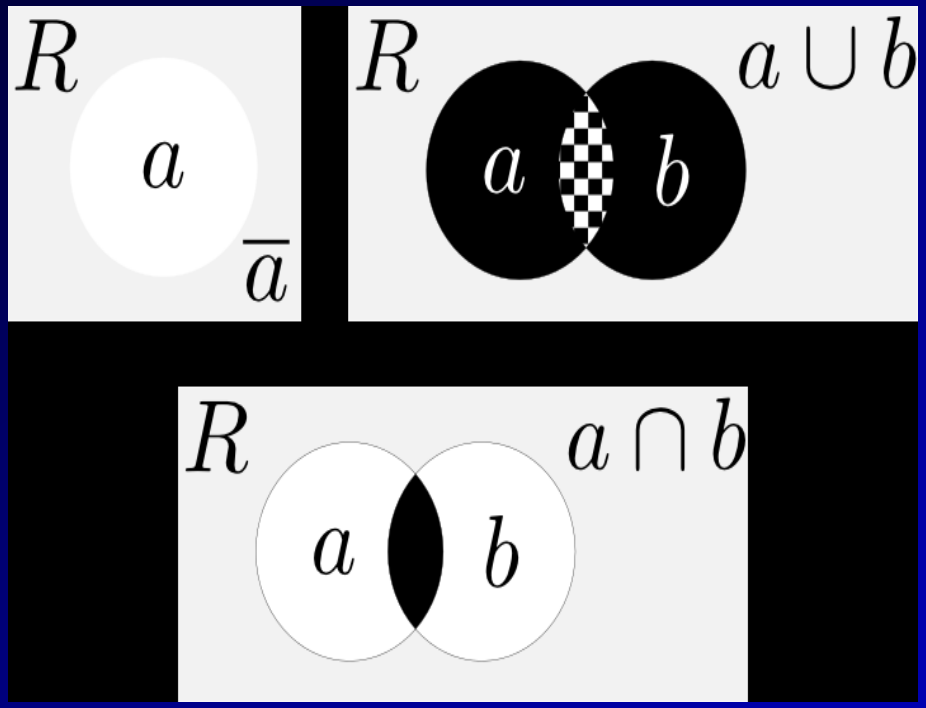
La fonction F(A,B) peut être la somme de A et B ou le résultat de la comparaison de A et B ou une autre fonction.

Une fonction logique de base est réalisée à l’aide des portes logiques qui permettent d'effectuer des opérations élémentaires.

Ces portes logiques sont aujourd'hui réalisées à l'aide de transistors.



1- Analogie ensembles et propositions logiques



Proposition = énoncé vrai ou faux

- a : les élèves sont présents
- b : le professeur est présent
- cours si a et b

Toute fonction logique, peut se décrire à l'aide des trois opérations de base :

- CONJONCTION ET (∧ ou .)
- DISJONCTION OU (v ou +)
- NÉGATION NON (¬ ou ¯)

a	b	¬ a	a∧b	a v b
V	V	F	V	V
V	F	F	F	V
F	V	V	F	V
F	F	V	F	F

George Boole, mathématicien anglais a voulu faire un lien entre la logique (étude de la validité du raisonnement) et la représentation symbolique utilisée en mathématique.

Plus tard, les travaux de Boole gagnent en intérêt lorsque Claude Shannon, fait le lien entre l'algèbre de Boole et la conception des circuits à relais.

Shannon montre que l'algèbre de Boole peut-être utilisée pour optimiser les circuits à relais.

L’algèbre de Boole est donc au cœur de la conception numérique.

Par convention, on associe généralement à l’état vrai d’une variable logique la valeur binaire 1 et la valeur 0 à l’état faux. C’est la logique positive. On peut aussi faire le contraire (c’est la logique négative) en associant la valeur 0 à l’état vrai et la valeur 1 à l’état faux. Dans ce cours, on travaillera toujours en logique positive, sauf mention contraire.

2 – Propriétés :

On considère les opérateurs { ¬ , ∧ , v } sur { 0, 1 }

par convention on écrit : a+b au lieu de a v b,

a.b pour a ∧ b,

$\bar{a}$ pour ¬ a

- Commutativité :

  a+b = b+a

  a.b = b.a

- Associativité :

  a+(b+c) = (a+b)+c

  a.(b.c) = (a.b).c

- Distributivité :

  $a.(b+c) = a.b + a.c$

  $a+(b.c) = (a+b).(a+c)$

- Idempotence :

  $a + a = a$

  $a \,.\, a = a$

- Absorption :

  $a + a \,.\, b = a$

  $a \,.\, (a + b) = a$

- Involution : $\bar{\bar{a}} = a$
- Elément neutre :

  $a + 0 = a$

  $a \cdot 1 = a$
- Elément absorbant :

  $a + 1 = 1$

  $a \cdot 0 = 0$
- Inverse :

  $a + \bar{a} = 1$

  $a \cdot \bar{a} = 0$

Exercices :

1°) Soit A = a + a.b

Démontrer que A = a

2°) Soit B = a . (a + b)

Démonter que B = a

3°) Soit C = a + $\bar{a}$. b

Démonter que C = a + b

4°) Soit D = (a +b) . (a + $\bar{b}$)

Démonter que D = a

3 - Table de vérité :

Une table de vérité est un tableau permettant de décrire toutes les possibilités de sorties en fonction des entrées.

On place donc les variables d'entrées dans les colonnes de gauche en les faisant varier de telle façon à couvrir l'ensemble des possibilités. La colonne de droite (ou les colonnes si la fonction a plusieurs sorties) décrit la sortie.

Exercice :

Reprendre les exercices avec la table de vérité

4 - Tableau de Karnaugh :

Le tableau de Karnaugh est un outil qui permet de simplifier des équations booléennes, avec moins de chances de se tromper qu'en utilisant les règles de l'algèbre de Boole classique.

Il s’utilise pour démontrer une égalité ou simplifier des expressions.

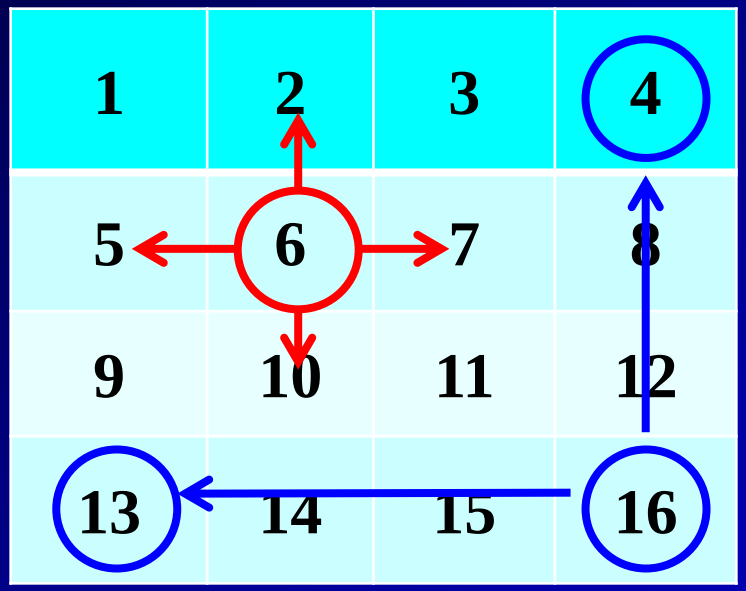
Pour prouver que deux expressions sont égales, on peut montrer que leurs diagrammes de Karnaugh sont identiques.

Le tableau de Karnaugh contient les mêmes informations qu'une table de vérité. La différence entre les deux représentations c'est qu'une table de vérité est écrite verticalement en une seule colonne, alors que le tableau de Karnaugh est étalé aussi bien en ligne qu'en colonne. Dans les deux cas, la table de vérité ou le tableau de Karnaugh indique l'état de la sortie (0 ou 1) en fonction de l'état des entrées logiques.

Simplification par le tableau de Karnaugh :

La méthode consiste à mettre en évidence, par un procédé graphique, tous les termes d'une fonction logique qui ne diffèrent que par l'état d'une seule variable (termes dits « logiquement » adjacents). Pour cela on réalise des groupements de cases adjacentes. Ces groupements de cases doivent être de taille maximale (nombre de cases max.) et doivent contenir un multiple de 2^n cases. On ne peut donc faire que des regroupements de 1, 2, 4, 8, 16, 32 cases etc.

Règle des adjacences



Le passage de la case **6** à l'une des quatre adjacentes ne modifie l'état que d'une seule variable.

On cesse d'effectuer les groupements lorsque tous les « 1 » appartiennent au moins à l'un d'eux.
Chaque regroupement donnera alors 1 terme dans l'équation logique finale, en inscrivant dans ce terme seulement les variables qui ne changent pas d'état sur l'ensemble du regroupement.
Il est fréquent qu'une case appartienne à plusieurs regroupements à la fois, afin que ces derniers soient le plus grand possible :
NB : il faut réaliser le minimum de regroupement.

Principe général :

Deux propositions sont égales si et seulement si leurs diagrammes de Karnaugh sont identiques.

Exercice 1 :

$F(a,b) = \bar{a}b + a\bar{b} + ab$

a \ b	0	1
0	1	0
1	1	1

$F(a,b) = a + \bar{b}$

Exercice 2 :

F(a,b,c) = ?

b \ ac	0	1	1	1
0	0	0	1	1

$F(a,b,c) = a+\overline{b}c$

Exercices : 2 variables ; 3 variables ; 4 variables

5 - Dualité de l'algèbre de Boole :

Toute expression logique reste vraie si on remplace le ET par le OU, le OU par le ET, le 1 par 0, le 0 par 1

Exemple :

$A + 1 = 1 \longrightarrow A \cdot 0 = 0$

$A + \overline{A} = 1 \longrightarrow A \cdot \overline{A} = 0$

6 - Théorème de DE MORGANE :

La somme logique complémentée de deux variables est égale au produit des compléments des deux variables.

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Le produit logique complémenté de deux variables est égale à la somme logique des compléments des deux variables.

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

Généralisation à N variables :

$$\overline{A+B+C+\ldots\ldots} = \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot \ldots\ldots$$

$$\overline{A \cdot B \cdot C \cdot \ldots\ldots} = \overline{A} + \overline{B} + \overline{C} + \ldots\ldots$$

7. Autres opérateurs logiques :

OU exclusif (XOR) ;

NON ET (NAND) ; NON OU (NOR)

III - ARCHITECTURE DE BASE D'UN ORDINATEUR

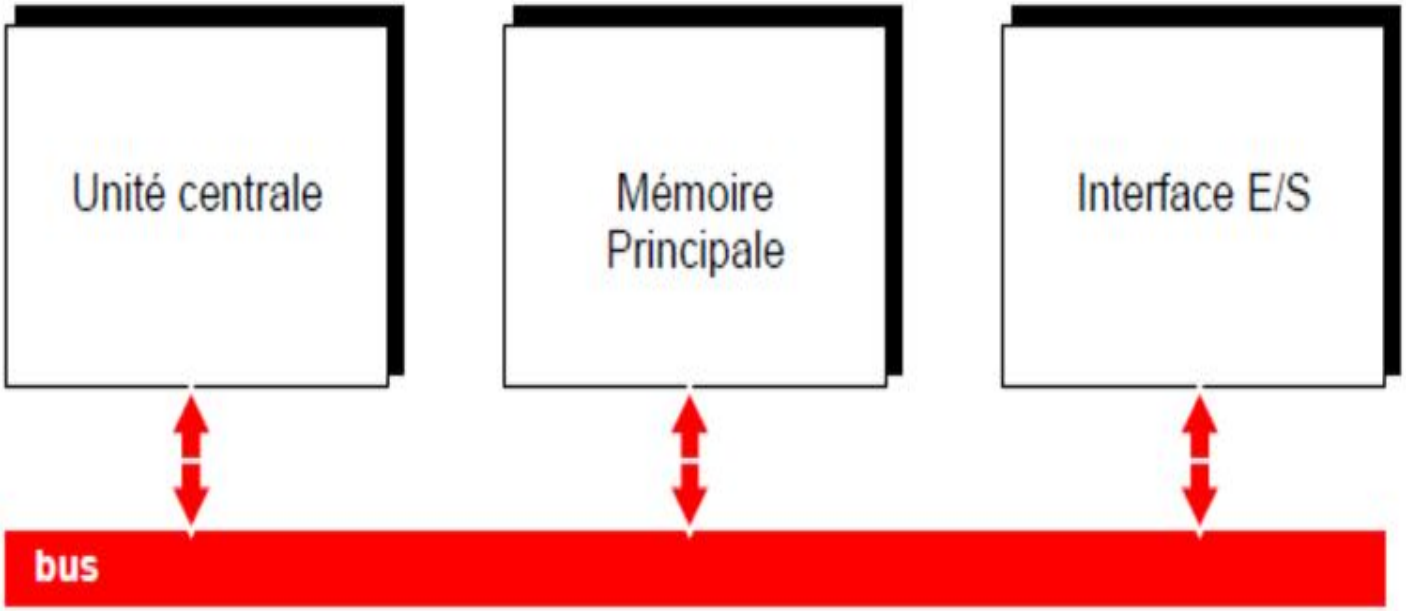
1- Modèle de VON NEUMANN

John Von Neumann est à l'origine d'un modèle de machine universelle de traitement programmé de l'information (1946). Cette architecture sert de base à la plupart des systèmes à microprocesseur actuel.

Elle est composée des éléments suivants :

- une unité centrale
- une mémoire principale
- des interfaces d'entrées/sorties

Les différents organes du système sont reliés par des voies de communication appelées **bus.**


Unité centrale
Mémoire Principale
Interface E/S
bus

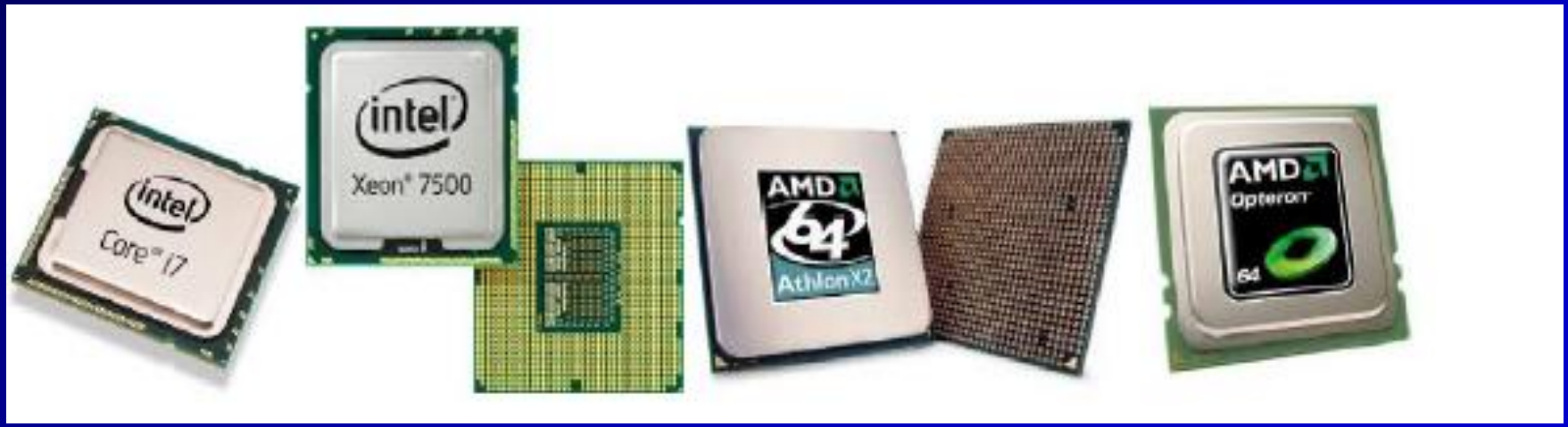
L'UNITÉ CENTRALE DE TRAITEMENT :

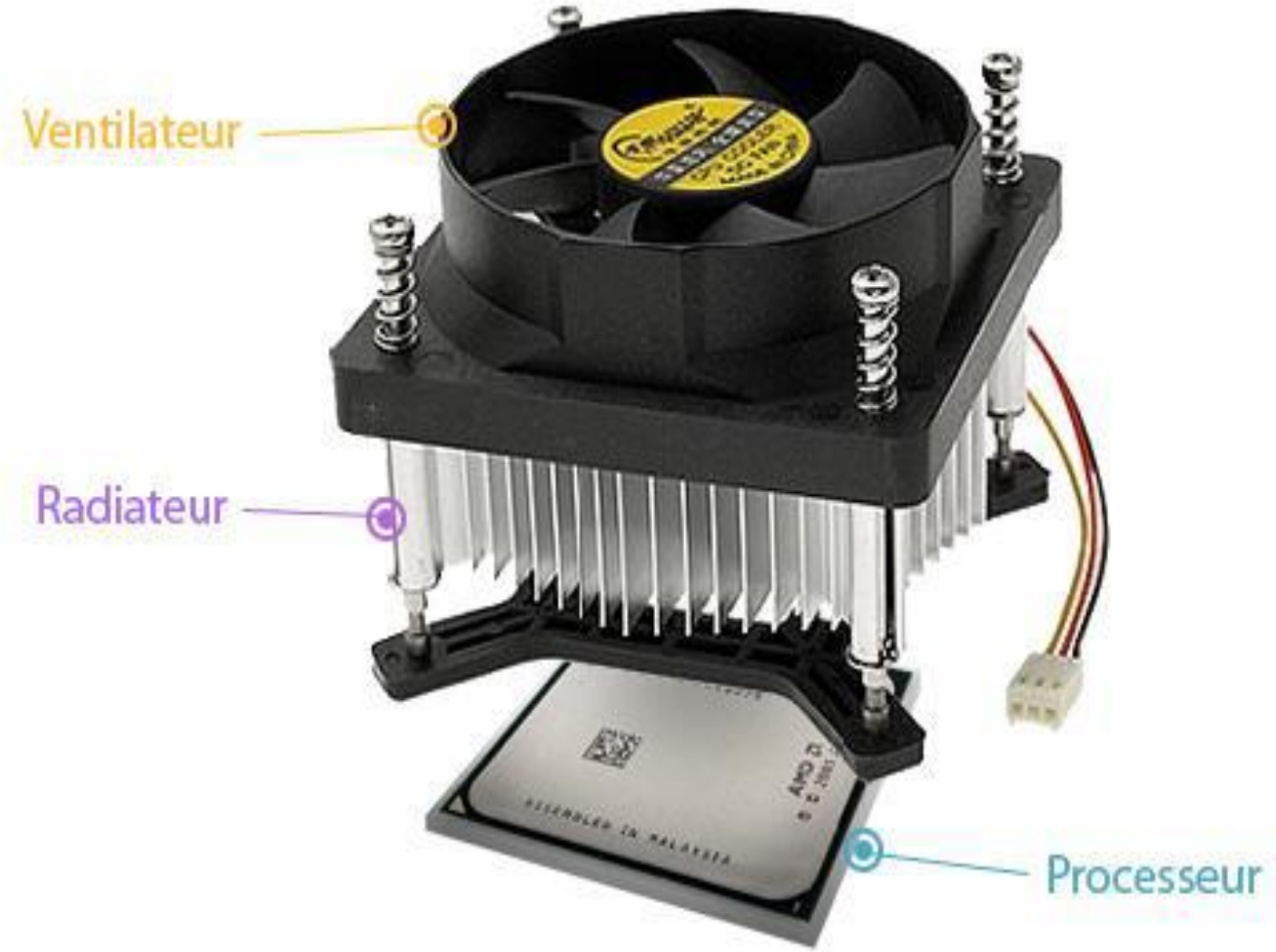
(CPU : Central Processing Unit) constituée :

- d'une Unité Arithmétique et Logique (UAL ou ALU) : c'est l'organe de calcul du calculateur
- d'une Unité de Contrôle (CU : Control Unit) : elle est chargée de commander et de gérer tous les différents constituants de l'ordinateur (contrôler les échanges, gérer l'enchaînement des différentes instructions, etc...)
- de Registres : zones de stockage des données de travail de l'UAL (opérandes, résultats intermédiaires) ;

Parfois appelé aussi CPU (Central **P**rocessing **U**nit), le processeur ou microprocesseur est le cerveau de l’ordinateur. C'est le composant le plus cher aussi.

Sa puissance dépend du nombre de cœurs du processeur, de la taille de sa mémoire cache, de l'architecture du processeur, et de sa fréquence.

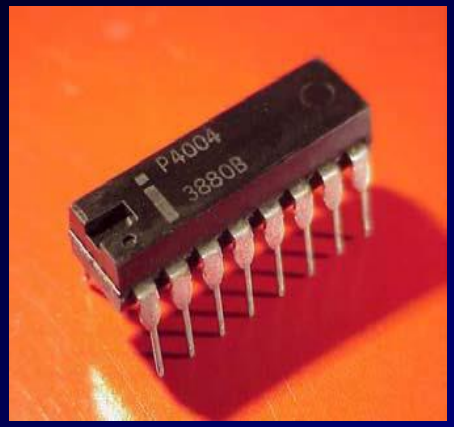



Ventilateur
Radiateur
Processeur

On caractérise le microprocesseur par :

- sa fréquence d'horloge : en MHz ou GHz
- le nombre d'instructions par secondes qu'il est capable d'exécuter : en MIPS
- la taille des données qu'il est capable de traiter : en bits (8, 16, 32, 64, 128 bits).

Toutes les activités du microprocesseur sont cadencées par une horloge.



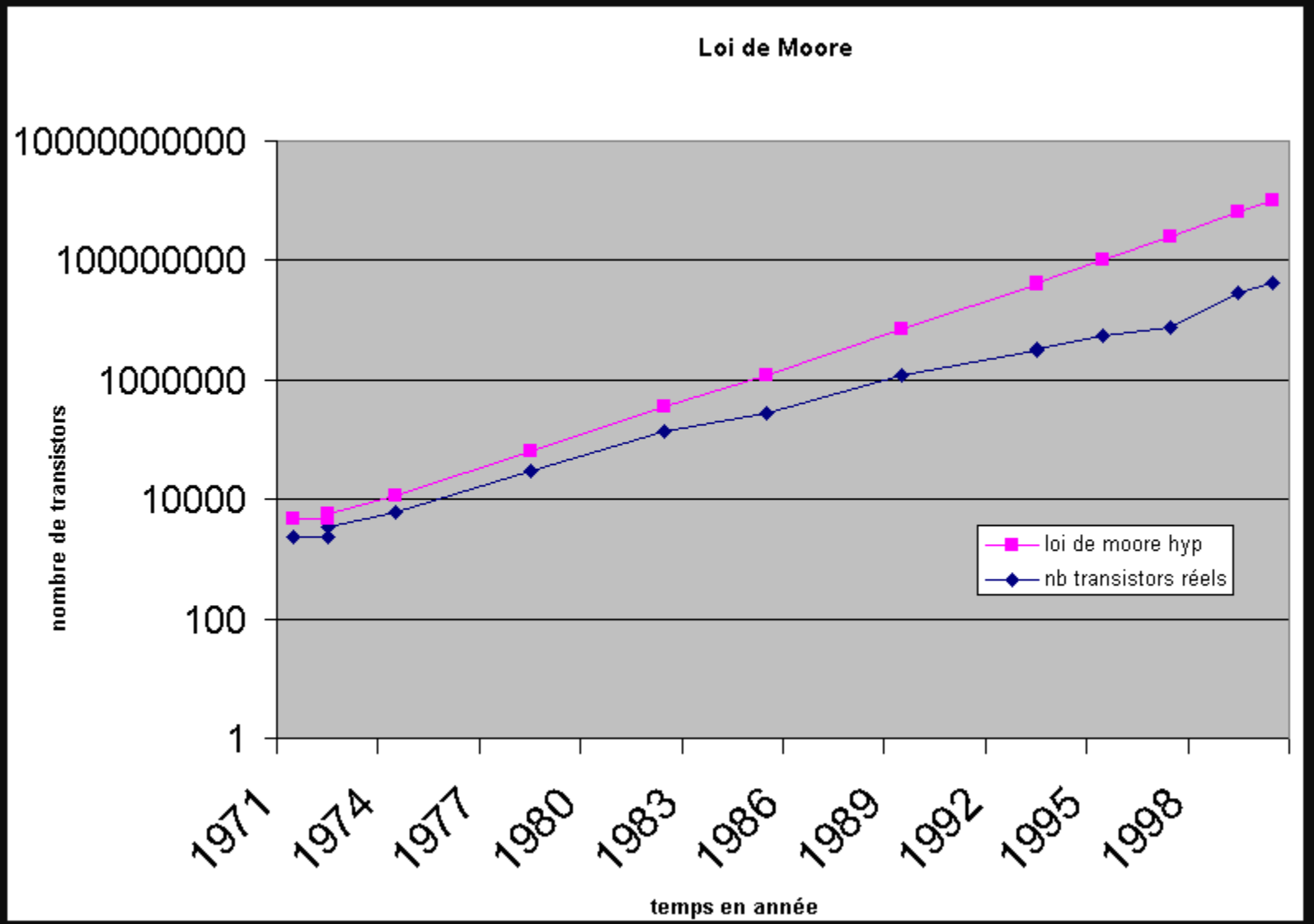
1971 : Intel 4004, premier microprocesseur (4 bits). fonctionnant à 740 Khz et intégrant 2300 transistors. 0,092 MIPS



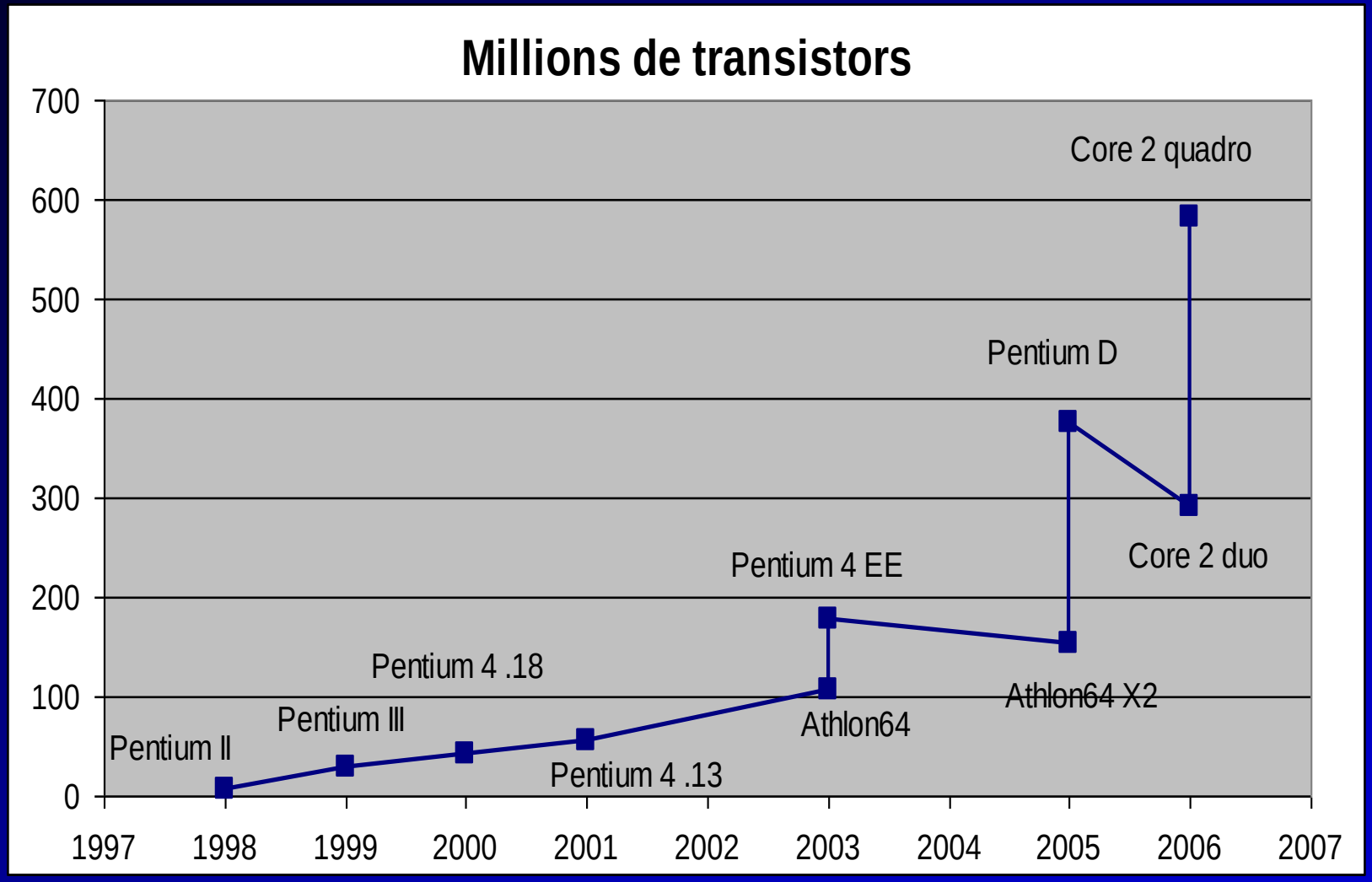
2011 : Intel Core i7 modèle 2600K (64 bits), 4 cœurs ; 1,4 milliards de transistors ; fonctionnant à 3,4 Ghz. 128300 MIPS

LOI DE MOORE :

À surface constante, le nombre de transistors double tous les 18 mois (1965).


Loi de Moore
10000000000
100000000
1000000
10000
100
1
nombre de transistors
1971
1974
1977
1980
1983
1986
1989
1992
1995
1998
temps en année
loi de moore hyp
nb transistors réels

Loi de Moore : quel avenir ?



2 -Architecture de base du microprocesseur :

- L'Unité Arithmétique et Logique (UAL, en anglais ALU -Arithmetic and Logical Unit), qui prend en charge les calculs arithmétiques élémentaires et les tests ;
- L'Unité de Contrôle ou séquenceur, qui permet de synchroniser les différents éléments du processeur ; en particulier, il initialise les registres lors du démarrage de la machine et il gère les interruptions.
- Les Registres, qui sont des mémoires de petite taille (quelques octets), suffisamment rapides pour que l'UAL puisse manipuler leur contenu à chaque cycle de l'horloge.

Les différents registres :

- L'accumulateur est destiné à contenir les données qui doivent être traitées par l'ALU ;
- Les registres généraux servent au stockage de résultats intermédiaires ;
- Les registres d'adresses servent à confectionner des adresses de données particulières, par exemple les registres de base et d'index qui permettent entre autre d'organiser les données en mémoire comme des tables indicées.

- Le registre d'instruction contient le code de l'instruction qui est traitée par le décodeur / séquenceur.
- Le compteur ordinal ou pointeur d'instruction ou Program Counter en Anglais (PC) contient en permanence l'adresse de la prochaine instruction à exécuter. Il génère les adresses des instructions à exécuter les unes à la suite des autres.
  Certaines instructions demandent quelques fois de changer le contenu du compteur ordinal pour faire une rupture de séquence c'est à dire un saut ailleurs dans le programme.

- Le registre d'état appelé parfois registre de condition contient des indicateurs appelés flags (drapeaux) et dont les valeurs (0 ou 1) varient en fonction des résultats des opérations arithmétiques et logiques.
- Le pointeur de pile ou stack pointer : ce type de registre, dont le nombre varie en fonction du type de processeur, contient l’adresse du sommet de la pile (ou des piles) ;

Jeu d’instructions :

Le jeu d’instructions décrit l’ensemble des opérations élémentaires que le microprocesseur pourra exécuter. Il détermine l’architecture du microprocesseur et notamment celle du séquenceur.

Actuellement l’architecture des microprocesseurs comprend deux grandes familles :

- L’architecture CISC (Complex Instruction Set Computer) ;
- L’architecture RISC (Reduced Instruction Set Computer)

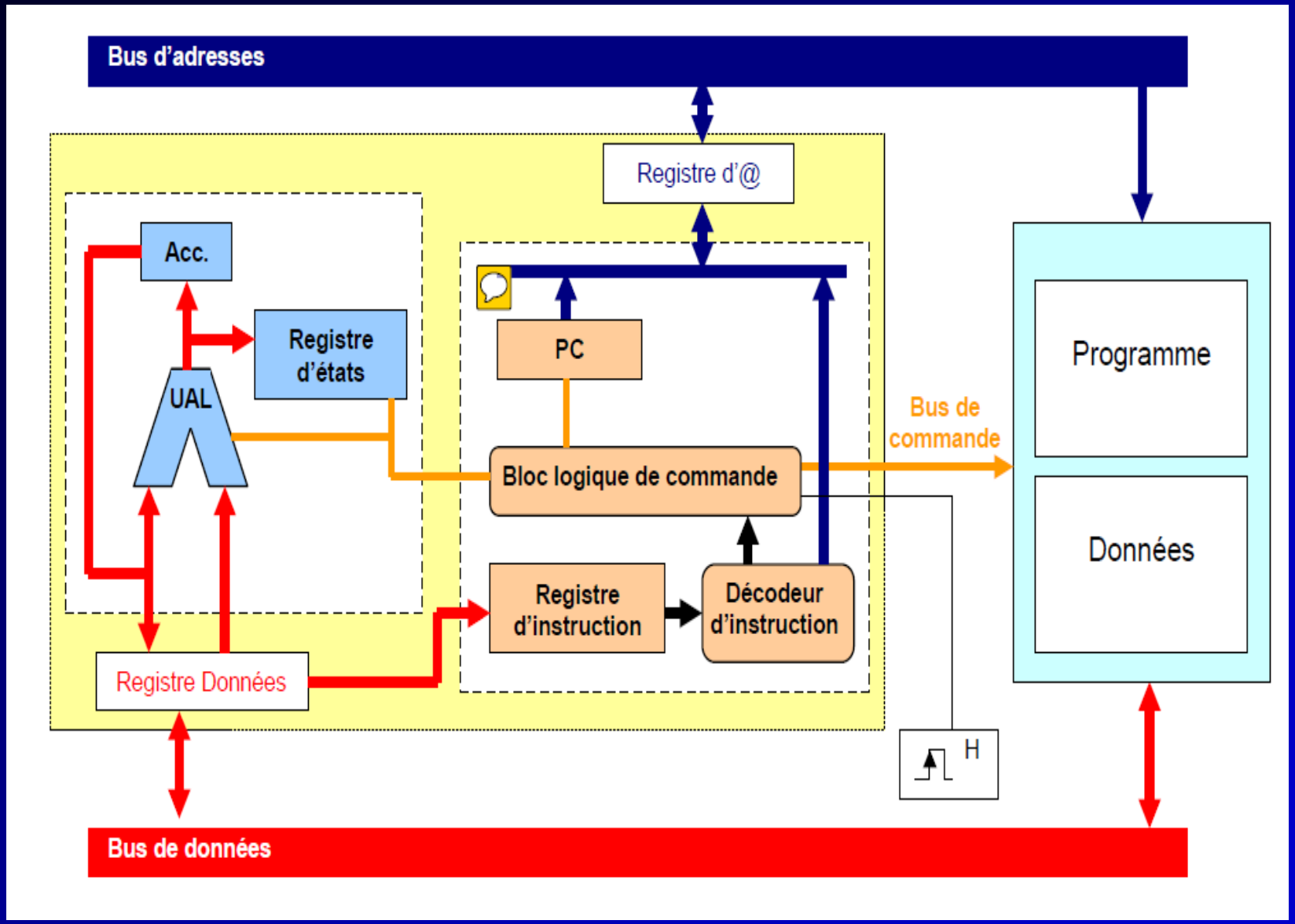
Type d'instructions :

Les instructions que l'on retrouve dans chaque microprocesseur peuvent être classées en 4 groupes :

- Transfert de données pour charger ou sauver en mémoire, effectuer des transferts de registre à registre, etc. …
- Opérations arithmétiques : addition, soustraction, division, multiplication
- Opérations logiques : ET, OU, NON, NAND, comparaison, test, etc. …
- Contrôle de séquence : branchement, test, etc. …

Les différentes étapes du déroulement d’une instruction :

- Recherche de l'instruction en mémoire ;
- Décodage de l’instruction
- Envoi des ordres de Traitement et des données à l’Unité de Traitement ;
- Préparation de l'instruction suivante.


Bus d'adresses
Registre d'@
Acc.
Registre d'états
UAL
PC
Bloc logique de commande
Bus de commande
Programme
Données
Registre d'instruction
Décodeur d'instruction
Registre Données
H
Bus de données

STRUCTURE DE LA MÉMOIRE PRINCIPALE :

Une mémoire est un circuit à semi-conducteur qui permet d'enregistrer, de conserver et de restituer des informations (instructions et variables).

Constituée principalement de :

- La mémoire vive -*Random Acces Memory* (RAM) ou mémoire vive.
- La mémoire morte - *Read Only Memory* (ROM) qui dans un PC contient les fonctions de base du BIOS

La mémoire principale contient les instructions des programmes en cours d'exécution et les données associées à ces programmes. Physiquement, elle se décompose en :

- Une mémoire morte (ROM = Read Only Memory) qui stocke les programmes et données de base de la machine. C'est une mémoire à lecture seule ;
- Une mémoire vive (RAM = Random Access Memory) chargée de stocker les données intermédiaires ou les résultats de calculs. On peut lire ou écrire des données dedans, ces données sont perdues à la mise hors tension ;

Une mémoire vive sert au stockage temporaire de données. Elle doit avoir un temps de cycle très court pour ne pas ralentir le microprocesseur.

Il existe deux grandes familles de mémoires RAM :

- les RAM statiques ;
- les RAM dynamiques.

Une mémoire idéale serait une mémoire de grande capacité, capable de stocker un maximum d'informations et possédant un temps d'accès très faible afin que le processeur puisse travailler rapidement sur ces informations.

Caractéristiques d’une mémoire :

La capacité : c’est le nombre total de bits que contient la mémoire. Elle s’exprime souvent en octet.

Le format des données : c’est le nombre de bits que l’on peut mémoriser par case mémoire. On dit que c’est la largeur du mot mémoire.

Le temps d’accès : c’est le temps qui s'écoule entre l'instant où a été lancée une opération de lecture / écriture en mémoire et l'instant où la première information est disponible sur le bus de données.

Le temps de cycle : il représente l'intervalle minimum qui doit séparer deux demandes successives de lecture ou d'écriture.

Le débit : c’est le nombre maximum d'informations lues ou écrites par seconde.

Volatilité : elle caractérise la permanence des informations dans la mémoire. L'information stockée est volatile si elle risque d'être altérée par un défaut d'alimentation électrique et non volatile dans le cas contraire.

Adressage mémoire :

Les mémoires informatiques sont organisées comme un ensemble de cellules pouvant chacune stocker une valeur numérique. Chaque cellule possède un numéro unique, appelé adresse, auquel les programmes peuvent se référer.

Toutes les cellules d'une mémoire contiennent le même nombre de bits.

Une cellule de n bits peut stocker 2^n valeurs numériques différentes.

Exercice 1 :

On considère une mémoire centrale de 2 Mbytes, où chaque byte est adressable séparément.

a) calculer l'adresse en octal, du 6è élément d'un tableau dont l'adresse du premier est 77_8 et dont tous les éléments sont composés de 16 bits.

b) calculer en décimal, le nb de bytes précédant l'adresse 77_8

c) calculer la taille de cette mémoire en l'exprimant en mots de 16 bits et en mots de 32 bits.

Exercice 2 :

Si le registre d'adresse d'une mémoire comporte 32 bits, calculer :

a) le nb de mots adressables si 1 mot = 1 byte
b) la plus haute adresse possible pour ces mots de 1 byte.
c) le nb de mots adressables si 1 mot = 32 bits
d) la plus haute adresse possible pour ces mots de 32 bits.

LES BUS :

Les différents composants sont reliés par des bus.

Un bus est un ensemble de fils qui assure la transmission du même type d'information.

On distingue trois types de bus véhiculant des informations en parallèle dans un système de traitement programmé de l'information :

- Bus de données : bus bidirectionnel qui assure l'échange des données entre le microprocesseur et les autres organes et inversement. Le nombre de fil caractérise la capacité de traitement du processeur.

- Bus d'adresses : bus unidirectionnel qui sélectionne l'adresse à laquelle on veut lire ou stocker une information. Le nombre de lignes de ce bus permet de connaître la capacité mémoire adressable.
- Bus de commande (contrôle) : véhicule les signaux de synchronisation des données sur les autres bus. Il est bidirectionnel.

LES INTERFACES D'ENTRÉES/SORTIES :

Elles permettent d'assurer la communication entre le microprocesseur et le monde extérieur (clavier, souris, écran, imprimante, modem, etc…).

Fonctionnement des Interfaces d'entrée/sortie

Chaque périphérique est relié au système par l'intermédiaire d'une interface (ou contrôleur) dont le rôle est de :

- Connecter le périphérique au bus de données
- Gérer les échanges entre le microprocesseur et le périphérique

Pour cela, l'interface est constituée par :

- Un registre de commande dans lequel le processeur décrit le travail à effectuer (sens de transfert, mode de transfert).

- Un ou plusieurs registres de données qui contiennent les mots à échanger entre le périphérique et la mémoire
- Un registre d'état qui indique si l'unité d'échange est prête, si l'échange s'est bien déroulé, etc. …

L'accès aux données de l'interface se fait par le biais d'un espace d'adresses d'entrées/sorties.

Cas particulier :

Carte graphique (carte vidéo, adaptateur graphique)

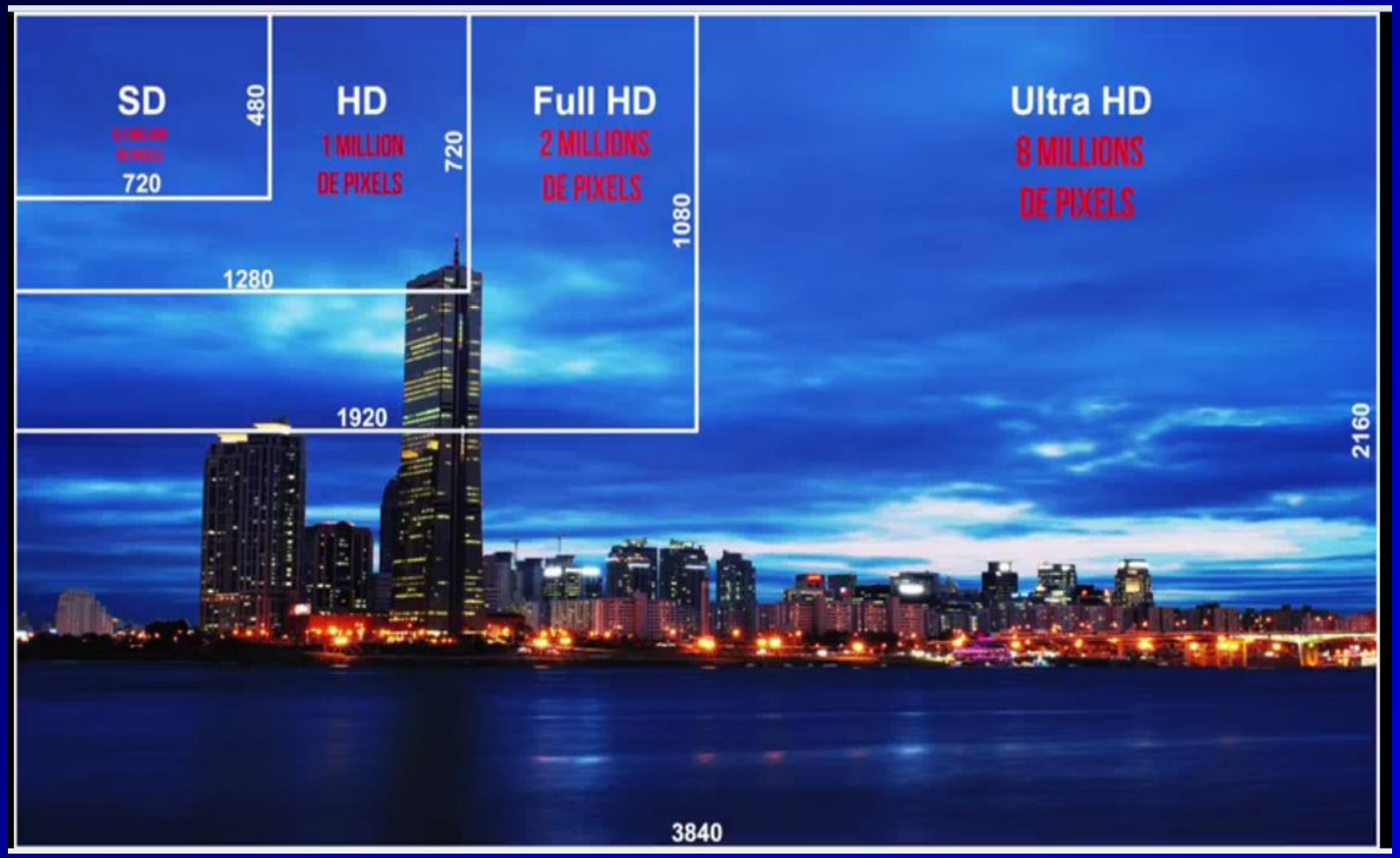
La carte vidéo est conçue pour permettre une communication de données entre le processeur et elle ainsi qu'entre elle et l'écran.

Sur un écran l'image est découpée en points élémentaires : pixels (picture's element).

Caractéristiques techniques de l'écran :

La définition d'écran, (couramment appelée à tort résolution d'écran), est le nombre de points ou pixels que peut afficher un écran. La définition est le produit du nombre de points selon l'horizontale par le nombre de points selon la verticale de l'affichage.

EXEMPLES :


SD
480
720
HD
1 MILLION
DE PIXELS
720
1280
Full HD
2 MILLIONS
DE PIXELS
1080
1920
Ultra HD
8 MILLIONS
DE PIXELS
2160
3840

Pour utiliser une définition donnée, il faut que la carte graphique soit capable d'en générer le signal vidéo, et que l'écran soit capable de l'afficher.
La définition d’une image est le nombre de points (ou pixels) que comporte une image numérique en largeur et en hauteur.

La taille de l’écran :

Elle se calcule en mesurant la diagonale de l'écran et est exprimée en pouces (un pouce équivaut à 2,54 cm).
Un écran d'une taille donnée peut afficher différentes définitions, cependant de façon générale les écrans de grande taille possèdent une meilleure définition.

Le pas de masque (dot pitch) :

C'est la distance qui sépare deux luminophores ; plus cette distance est petite plus l'image est précise. Ainsi un pas de masque inférieur ou égal à 0,25 mm procurera un bon confort d'utilisation, tandis que les écrans possédant des pas de masque supérieurs ou égaux à 0,28 mm seront à proscrire.

La taille de l'écran et son pas (pitch) définissent la définition maximale que l'on peut atteindre sans perte de qualité en ce qui concerne l'image.

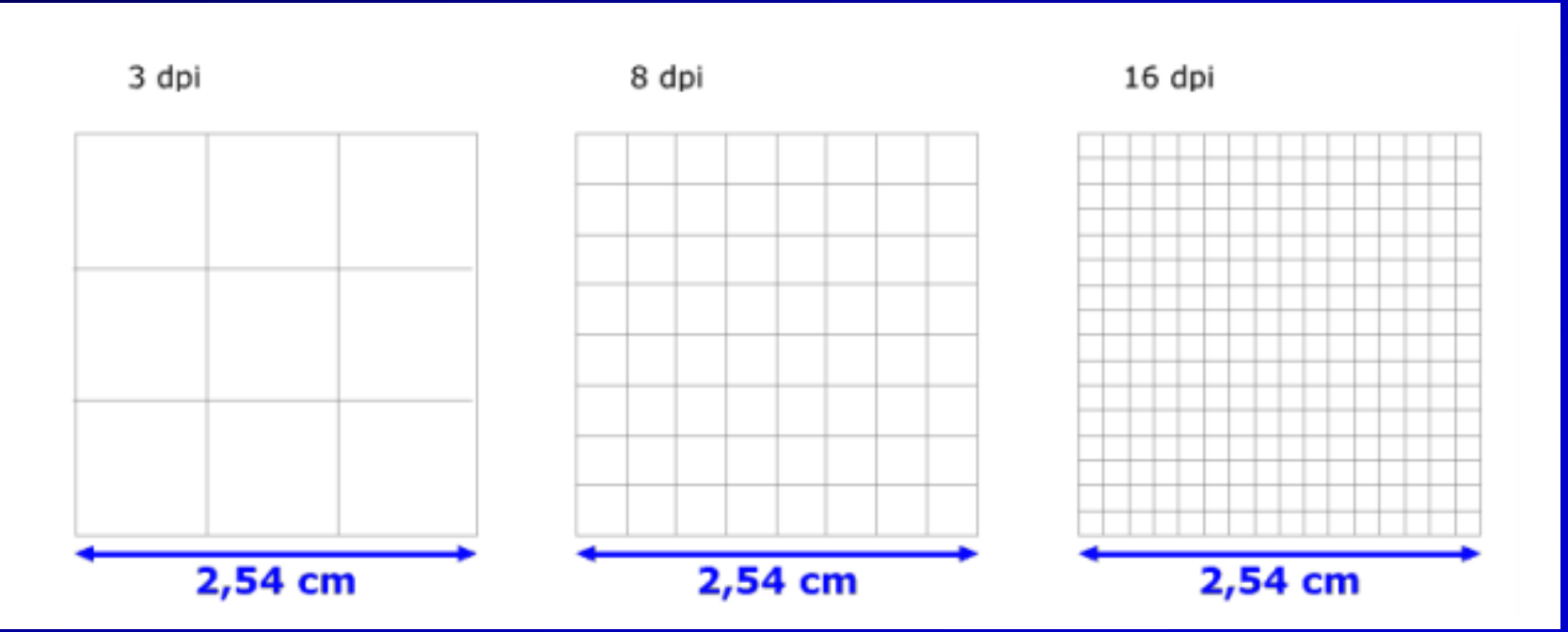
Diagonale	Équivalent	Définition
15"	38 cm	800x600
17"	43 cm	1024x768
19"	48 cm	1280x1024
21"	53 cm	1600x1200

Pour un **appareil photo numérique**, une **imprimante** ou un **scanner**, la définition indique le nombre de points pouvant être numérisés ou imprimés.

La résolution :

Elle détermine le nombre de pixels par unité de surface (pixels par pouce ou Points par pouce **PPP** ou Dots Per Inch **DPI** en anglais). Une résolution de 300 dpi signifie 300 colonnes et 300 rangées de pixels sur un pouce carré ce qui donnerait donc 90000 pixels sur un pouce carré.
La résolution définit la netteté et la qualité d'une image. Plus la résolution est grande (plus il y a de pixels dans une longueur de 1 pouce), plus l' image est précise dans les détails.

La résolution est un rapport de densité, elle exprime le nombre de pixels qui sont affichés dans un pouce (2,54cm) Pour des raisons techniques liées à la taille des composants électroniques des pixels, la majorité des écrans affichent 72 px par pouce, contrairement aux imprimantes qui sont capables d'imprimer des points bien plus petits (la norme des imprimeurs est 300dpi).



La couleur d'un pixel est constituée en mélangeant 3 couleurs : le rouge, le vert et le bleu : système RVB (RGB en anglais)

2 codages possibles des couleurs :

- Vraies couleurs (true color) : codage de la valeur de chaque composante sur un certain nombre de bits. Ex : 8+8+8 = 24 bits
- Palette : un nombre de couleurs déterminées et numérotées puis stockées dans une table.

Taille du codage :

- 2 couleurs : 1 bits par pixel (mémoire vidéo = plan de bits : bitmap)
- 4 couleurs : 2 bits par pixel
- 16 couleurs : 4 bits par pixel
- 256 couleurs : 8 bits par pixel
- 65 536 couleurs : 16 bits par pixel
- 16 777 216 couleurs (16,8 millions) : 24 bits par pixel
- 4 294 967 296 couleurs (4,3 milliards) : 32 bits

NB:

En vraies couleurs sur 24 bits on code 16,8 millions de couleurs : ceci suffit à représenter toutes les nuances décelables par l'œil humain (inutile d'utiliser plus de 8 bits par composantes).

Rapports entre les pixels, définition et résolution d'une image :

Taille d'impression (en pouce)=Définition / Résolution
Taille d'impression (en cm)= (Définition/Résolution)x 2.54

Exemple :

Soit une image qui a une définition de 1200x600 pixels, et une résolution de 300 dpi (donc 300 pixels par pouce) et nous voulons calculer la taille d'impression.

(1200 px de largeur / 300 dpi) x 2,54 = 10,16 cm (600 px de hauteur / 300 dpi) x 2,54 = 5,08 cm

On pourra donc imprimer l'image, sans perte de qualité en 10,16x5,08 cm

Astuce :

méthode approximative : diviser la définition en 300 dpi par 100 -------> 1200:100= 12

-------> 600 :100= 6

Capacité de la mémoire vidéo :

Ex : 1024 x 768 en 256 couleurs : 1024 * 768 * 8
= 6 291 456 bits = 768Ko

	Nombre de couleurs			
Résolution	16	256	65536	16,8M
640 x 480	150Ko	300Ko	600Ko	900Ko
800 x 600	235Ko	469Ko	938Ko	1,37Mo
1024 x 768	384Ko	768Ko	1,5Mo	2,25Mo
1280 x 1024	640Ko	1,25Mo	2,5Mo	3,75Mo
1600 x 1200	938Ko	1,83Mo	3,66Mo	5,49Mo

Pour gérer un affichage de qualité (1600 x 1200 en vraies couleurs) il faut déjà 6Mo de mémoire vidéo uniquement pour une image !

Les techniques d'échange de données :

Avant d'envoyer ou de recevoir des informations, le microprocesseur doit connaître l'état du périphérique pour que la transmission se fasse correctement. Il existe 2 modes d'échange :

- Le mode programmé par scrutation ou interruption où le microprocesseur sert d'intermédiaire entre la mémoire et le périphérique
- Le mode en accès direct à la mémoire (DMA) où le microprocesseur ne se charge pas de l'échange de données.

Types de liaisons :

Les systèmes à microprocesseur utilisent deux types de liaison différentes pour se connecter à des périphériques :

- liaison parallèle
- liaison série

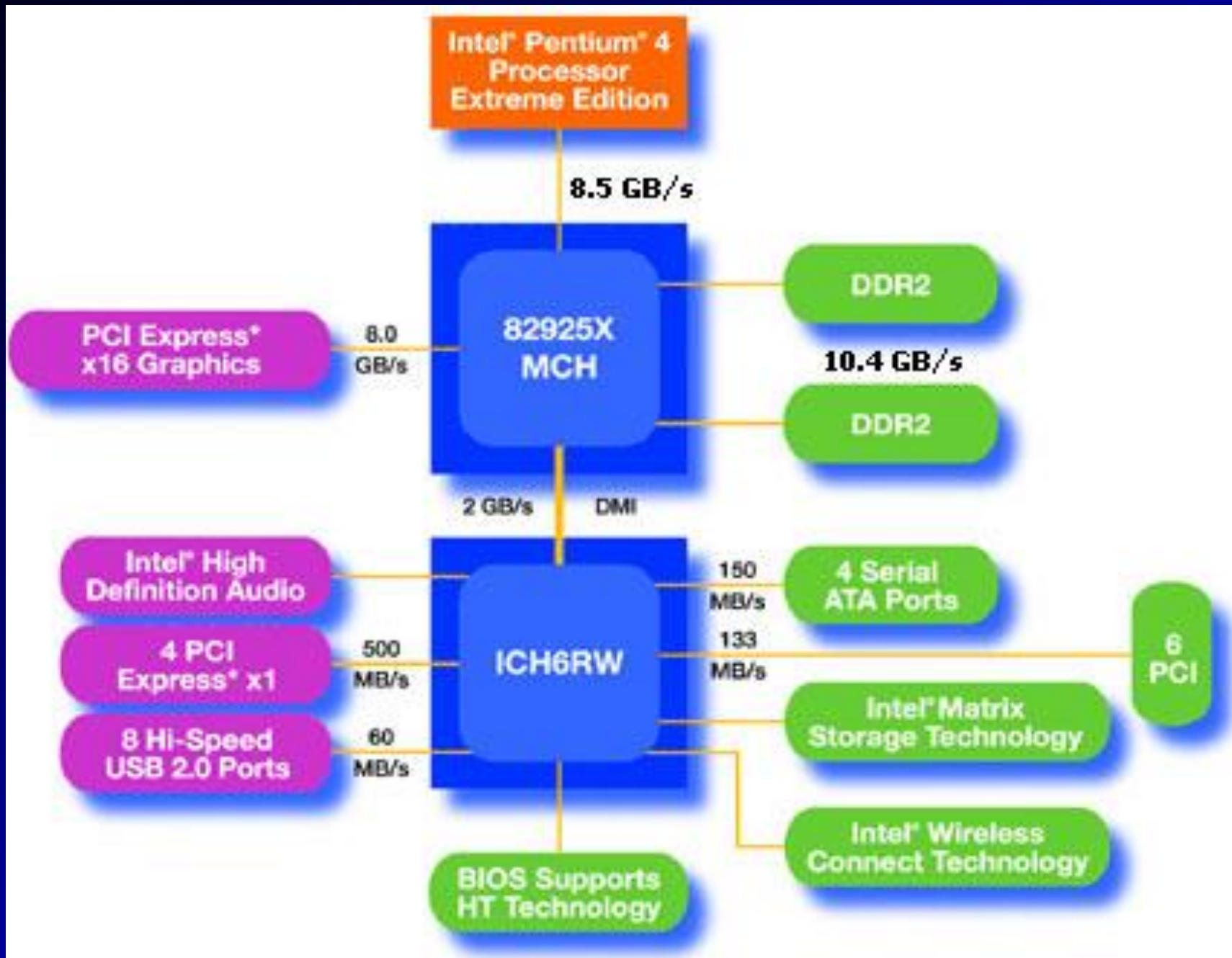
On caractérise un type de liaison par sa vitesse de transmission ou débit (en bit/s).

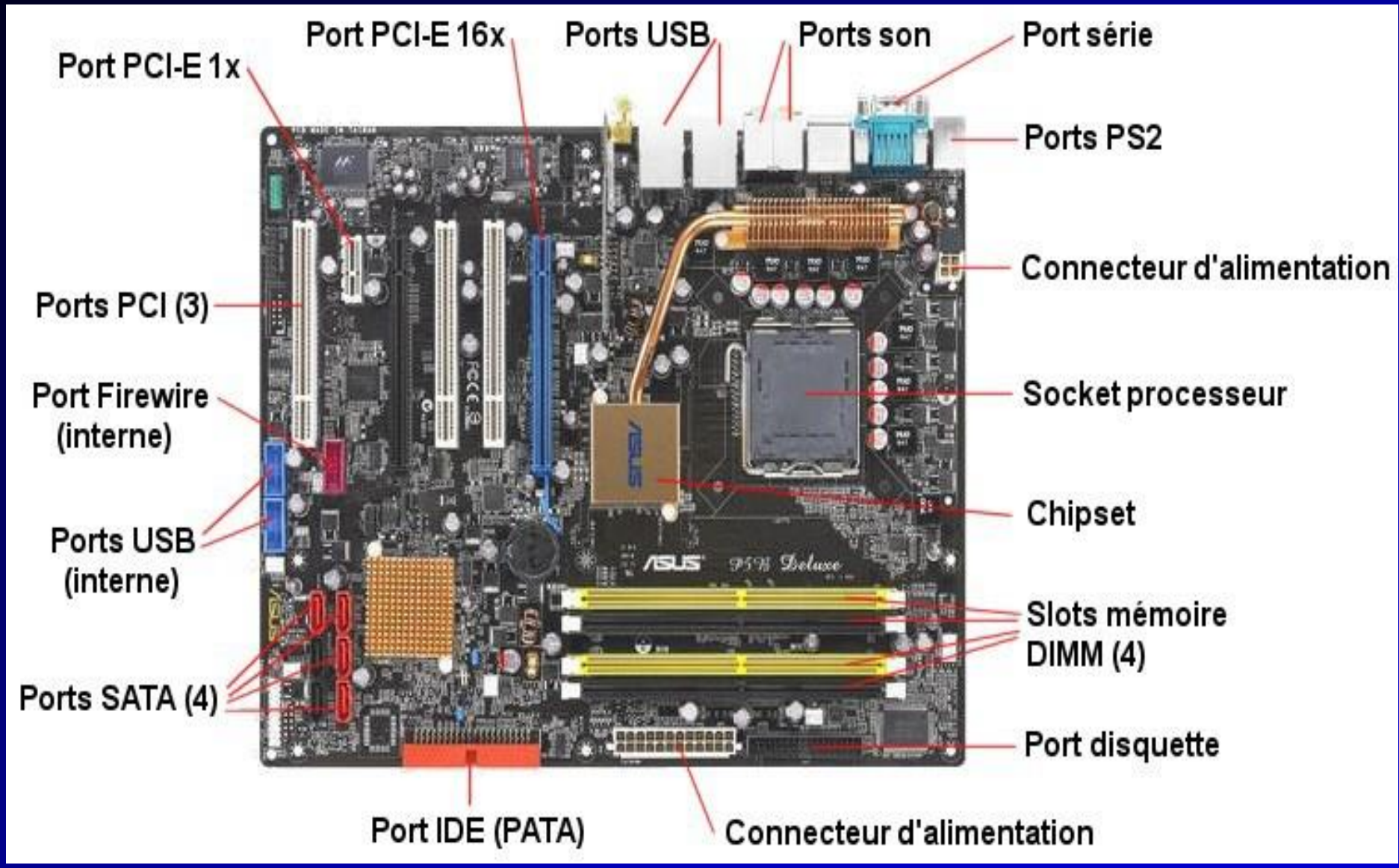
EXEMPLES

3 - APPLICATION – L'ORDINATEUR PERSONNEL :

Le système nerveux de l'ordinateur personnel (PC : Personal Computer) est la carte mère. Elle permet d'interconnecter tous les composants de l'ordinateur. Son choix conditionne les possibilités d'extension de l'ordinateur. Une carte mère comporte toujours les éléments suivants :

- Un chipset
- Un BIOS (mémoire de démarrage du PC)
- Une horloge interne
- Un bus système


Intel® Pentium® 4 Processor Extreme Edition
8.5 GB/s
82925X MCH
DDR2
10.4 GB/s
DDR2
PCI Express* x16 Graphics
8.0 GB/s
2 GB/s
DMI
ICH6RW
Intel® High Definition Audio
4 PCI Express* x1
500 MB/s
8 Hi-Speed USB 2.0 Ports
60 MB/s
150 MB/s
4 Serial ATA Ports
133 MB/s
6 PCI
Intel® Matrix Storage Technology
Intel® Wireless Connect Technology
BIOS Supports HT Technology


Port PCI-E 1x
Port PCI-E 16x
Ports USB
Ports son
Port série
Ports PS2
Connecteur d'alimentation
Ports PCI (3)
Port Firewire (interne)
Socket processeur
Chipset
Ports USB (interne)
Slots mémoire DIMM (4)
Ports SATA (4)
Port disquette
Port IDE (PATA)
Connecteur d'alimentation

Le socket reçoit le processeur ;

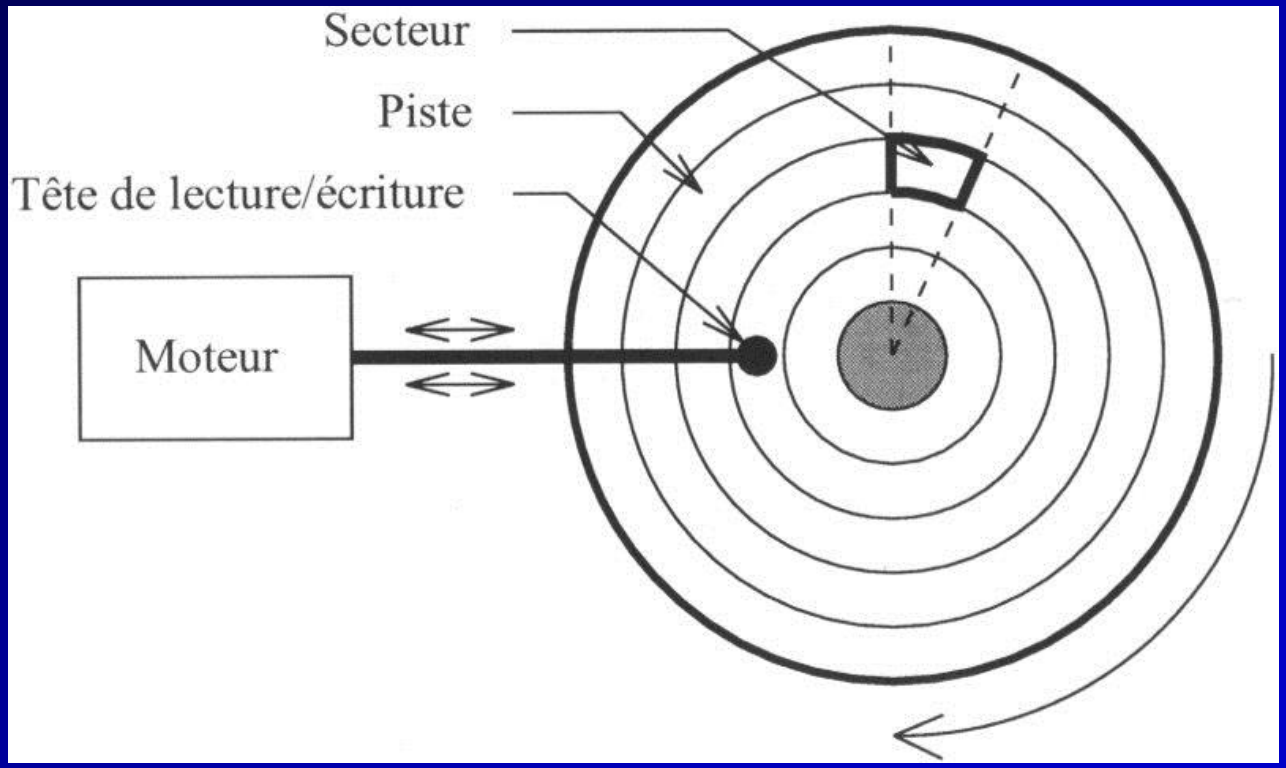
Le chipset gère les flux de données entre le processeur et les autres composants via le Front Side Bus (FSB) :

- Le Northbridge gère les relations avec la mémoire et les ports graphiques AGP (AGP - Advanced Graphics Port)
- Le Southbridge gère les flux plus lents. Mais de plus en plus de leurs fonctionnalités tendent à être incorporées dans le processeur

Les slots mémoire reçoivent les barrettes mémoire.

Le Port PCI (Peripheral Component Interconnect) : permet de connecter des cartes d'extension sur la carte mère via un bus PCI (cartes réseau, son, graphique) débit théorique 8 Go/s en 32 bits
Le Ports SATA (Serial Advanced Technology Attachement) pour connecter une mémoire de masse (typiquement un disque dur ou un lecteur de CD/DVD) à la carte mère : 6 Gbits/s pour SATA 3
Les Ports série : transmettent des données en série (bit à bit), dont notamment le Universal Serial Bus (USB) : débit 4,8 Gbit/s pour USB 3

Autres périphériques : connectés en IDE ou SATA disque dur, lecteur/graveur DVD, disque externe en USB : (pour le stockage).

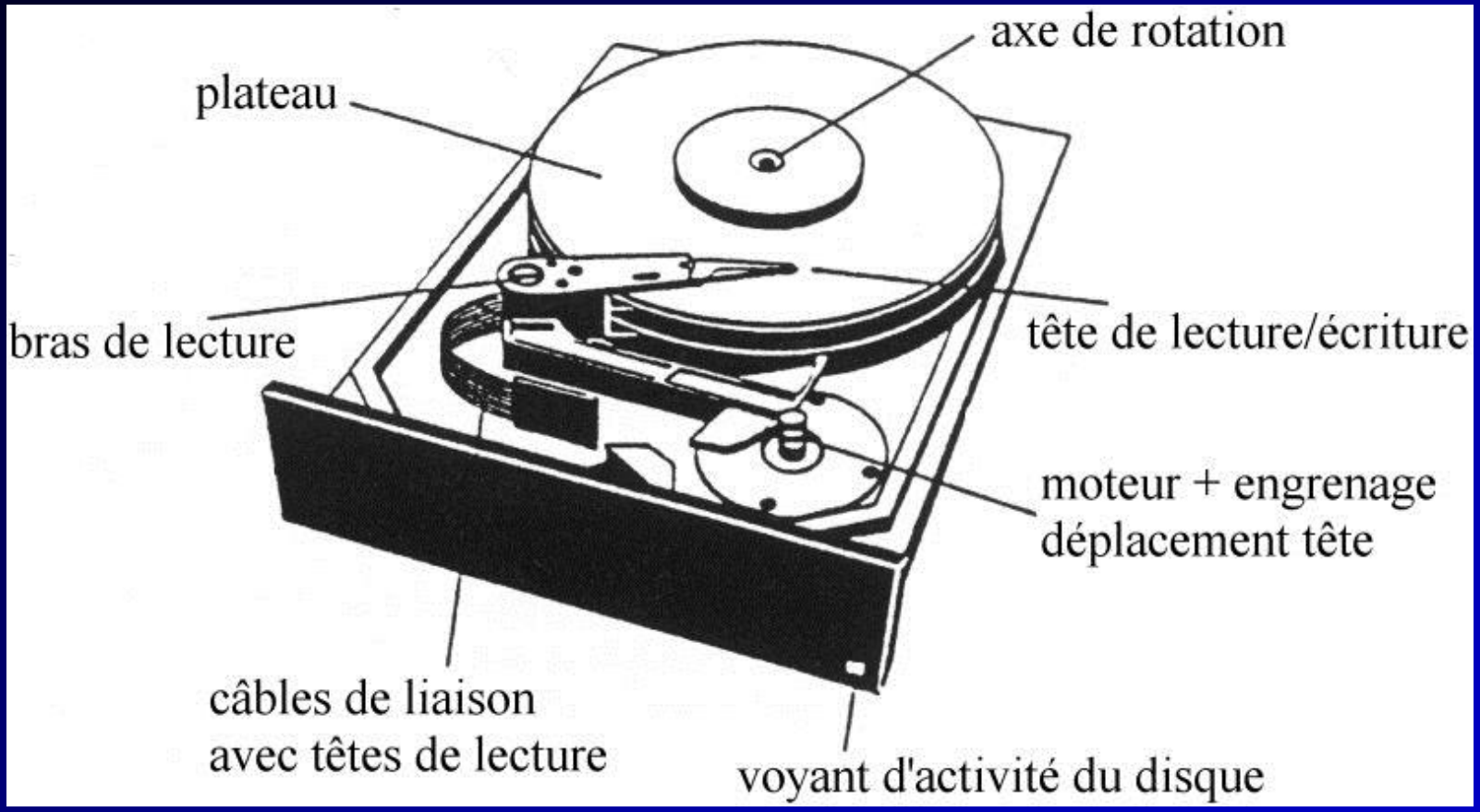


Structure physique d’un disque dur

Le formatage logique est effectué après le formatage physique. Il permet de réserver les premiers secteurs pour y stocker des programmes ou des informations systèmes et de regrouper les secteurs en clusters (blocs logiques). On y trouve :

- une zone d'amorçage qui peut contenir le programme de lancement (si c'est un disque système) ; une table d'allocation de fichiers ; le répertoire racine.

le reste du disque sert au stockage des données



Un disque dur est composé d'un certain nombre de plateaux rigides fixés sur un même axe de rotation.

IV - INTRODUCTION AU LANGAGE MACHINE

1- Langages informatiques :

Un langage informatique est un langage destiné à décrire l'ensemble des actions consécutives qu'un ordinateur doit exécuter pour réaliser une tâche.

A la manière d’une langue naturelle, Un langage informatique est construit à partir d'une grammaire formelle, qui inclut des symboles et des règles syntaxiques, auxquels on associe des règles sémantiques. Ces éléments sont plus ou moins complexes selon la capacité du langage.

Les langages de programmation permettent de décrire d'une part les instructions qui seront exécutées par le processeur, et d'autre part d'indiquer comment les manipulations sont effectuées et selon quels algorithmes.
Ils servent de moyens de communication par lesquels le programmeur communique avec l'ordinateur, mais aussi avec d'autres programmeurs.
une instruction machine est une opération élémentaire qu'un programme demande à un processeur d'effectuer. C'est l'ordre le plus basique que peut comprendre un ordinateur.

La collection d'instructions machine qui peuvent être données à un processeur est son jeu d'instructions. Les instructions machine sont codées en binaire. Un champ de l'instruction appelé « code opération » ou « opcode » désigne l'opération à effectuer. Puisque sa valeur numérique n'a pas de sens pour les humains, le programmeur utilise une abréviation désignant le code opération fourni par le langage assembleur pour ce processeur.

2 - Ecriture des instructions en langage symbolique

Voici un programme en langage machine 80x86, implanté à l'adresse 0100H :

A1 01 10 03 06 01 12 A3 01 14

Ce programme additionne le contenu de deux cases mémoire et range le résultat dans une troisième.
Afin de pouvoir écrire (et relire) des programmes en langage machine, on utilise une notation symbolique appelée langage assembleur.

Parmi les jeux d'instructions principaux on a :

- x86 ;
- PowerPC ;
- SPARC ;
- ARM ;
- Alpha ;
- VAX ;
- 68000 ;
- MIPS. Le jeu d'instructions précise non seulement les registres et instructions supportées par le processeur, mais aussi la façon dont ces instructions et les opérandes sont représentés en mémoire.

Taille des instructions :

La taille d'une instruction dépend de l'architecture de la plateforme, mais elle est généralement comprise entre 4 et 64 bits.

Exemple :

```
LOAD  R0, B        ; copie le contenu de l'adresse B
                     dans le registre R0
ADD   R1, R0, C    ; R1 = R0 + C
STORE R1, A        ; stocke la valeur de R1 à
                     l'adresse A
```

INSTRUCTIONS LES PLUS FRÉQUENTES :

- Instructions de transfert
- Instructions arithmétiques
- Instructions logiques
- Instructions d'entrées/sorties
- Instructions de branchement
- Instructions diverses

Instructions du 8086 / 8088 :

Nom	Sens	Traduction	Notes
AAA	**A**SCII **A**djust AL after **A**ddition	Ajuste le registre AL après addition (mode ASCII)	Utilisé avec le codage BCD non compacté
AAD	**A**SCII **A**djust AX after **D**ivision	Ajuste le registre AX après division (mode ASCII)	Utilisé avec le codage BCD non compacté, bogué dans le jeu d'instruction original, mais « réparé » dans le NEC V20, causant de nombreuses incompatibilités
AAM	**A**SCII **A**djust AX after **M**ultiplication	Ajuste le registre AX après multiplication (mode ASCII)	Utilisé avec le codage BCD non compacté
AAS	**A**SCII **A**djust AL after **S**ubtraction	Ajuste le registre AL après soustraction (mode ASCII)	Utilisé avec le codage BCD non compacté
ADC	**Ad**d with **C**arry	Ajoute deux entiers, plus le drapeau de retenue	Le résultat remplace le premier opérande
ADD	**Add**	Ajoute deux entiers	Le résultat remplace le premier opérande
AND	Logical **AND**	Effectue un ET logique des opérandes	Le résultat remplace le premier opérande
CALL	**Call** Procedure	Appelle une procédure	
CBW	**C**onvert **B**yte to **W**ord	Convertit un octet en mot	Le registre AL est étendu à AX
CLC	**Cl**ear **C**arry Flag	Met le drapeau de retenue à zéro	
CLD	**Cl**ear **D**irection Flag	Met le drapeau de direction à zéro	
CLI	**Cl**ear **I**nterrupt Flag	Met le drapeau d'interruption à zéro	

Quel est l’intérêt de programmer en assembleur de nos jours ? Pour une programmation standard, aucun : en effet, les compilateurs produiront toujours un code plus optimisé que celui qu'on écrirait à la main.

Utile dans certains cas précis :

- Programmation bas-niveau de pilotes de périphériques
- Utilisation de processeurs dans des cartes électroniques
- Systèmes d’exploitation

LE PARADOXE DU MENSONGE

A la frontière de ce Royaume, les soldats demandent à chaque visiteur :

Pourquoi venez-vous ici ?

Si le voyageur dit la vérité, il est décapité, s'il ment il est pendu.

Un jour, un voyageur répond :

Je viens ici pour être pendu.

En quoi cette réponse est-elle judicieuse ?

LE PARADOXE DU MENSONGE

Un homme qui vient de mourir arrive dans une salle où se trouvent deux portes. L'une mène au paradis, l'autre en enfer. Devant chaque porte se trouve un gardien qui ne sait dire que **OUI** ou **NON**. L'homme sait qu'un des deux gardiens ment toujours et que l'autre dit toujours la vérité. Mais il ne sait pas lequel ment ni lequel dit la vérité.
Il a droit à une seule question.
Quelle question doit-il poser ….. ?